

Tide retrospective: *the first consumer rewired onto `lean-common`*

Claude Opus 4.8 (1M ctx), with Daniel Murfet

2026-06-04

Abstract

A process tide, not a mathematical one. This session promoted four reusable cores into the shared `lean-common` library; the promotions were deliberately *additive* (new files nothing imported yet) so they could land without disturbing concurrently-worked consumer repos. This tide closes the loop for the first consumer. Laplace’s scaled weighted AM–GM bound $t|x| \leq \frac{tc}{2}x^2 + \frac{t}{2c}$, `amgm_t_abs_x`, was transplanted verbatim into `Common/AmGm.lean` a few tides ago; here we add `lean-common` as a git dependency, `import Common.AmGm`, delete the local copy, and qualify the two call sites. The point of the tide is the *mechanism*: it validates the dependency-add path end-to-end (clean `lake update`, no transitive-Mathlib conflict, 8306-job build green) so that the remaining consumers — `modes-lean` and `threepoint` — follow a proven recipe. No `sorry/axiom/native_decide`.

1 Setting

The session’s through-line was a linear-algebra arc — real-matrix SVD, the Moore–Penrose pseudo-inverse, Eckart–Young–Mirsky — traced across three Lean repos. Along the way three reusable cores proved themselves promotable: the Moore–Penrose-inverse predicate and its uniqueness API (used in both `threepoint` with the conjugate-transpose `H` and `modes-lean` with the real transpose `T`), the weighted “top-*k*” / water-filling majorization inequality (the combinatorial core of the Frobenius Eckart–Young), and the AM–GM domination bound from Laplace’s anharmonic regularity proof. The promotion test we settled on: *a lemma belongs in `lean-common` iff its own statement and proof mention nothing paper-specific* — not its home file’s imports. `amgm_t_abs_x` is the clean exemplar: it lived in a file that imports `Threepoint` and the rest of the Laplace stack, yet the lemma itself is a one-line scalar inequality with a four-line “complete-the-square” proof, portable verbatim.

Promotions were kept additive on purpose. A new file in `lean-common` that no consumer imports yet cannot break a consumer mid-concurrency; the disruptive half — adding the dependency and dropping the local copy — was deferred to a deliberate rewiring step. This is that step, for the first consumer.

2 The thing done

Point Laplace at `lean-common`’s `Common.AmGm.amgm_t_abs_x` and retire its local duplicate, leaving the build green and sorries-free.

Concretely: `lakefile.toml` gains a `[[require]]` on `lean-common` pinned to commit 511ee60; `AnharmonicGibbsRegularity.lean` gains `import Common.AmGm` and loses

its local theorem `amgm_t_abs_x`; the two call sites (one in that file at line 229, one in `AnharmonicPartitionDerivGeneralH.lean` at line 66) are requalified from the bare name to `Common.AmGm.amgm_t_abs_x`. The two theorems are textually identical — one was transplanted from the other — so the substitution is behaviour-preserving and the `lake build` *is* the proof.

3 How it works (the dependency mechanics)

Lake resolves a git `[[require]]` by cloning the dependency at the pinned `rev`, reading *its* `lake-manifest.json` to learn the transitive closure, and unifying that closure with the consumer’s own. The one real hazard is a *Mathlib pin mismatch*: if `lean-common` pinned a different Mathlib commit than `laplace`, the two would fight and the build would not resolve. Both are pinned at `8a178386` (the `v4.29.0` tag), so `lake update lean-common` unified cleanly and no Mathlib was re-fetched.

The second subtlety was invisible until the build: only *one* of the two call sites lives in the file we edited. The other, in `AnharmonicPartitionDerivGeneralH.lean`, had been consuming the local `amgm_t_abs_x` *transitively* — its import chain runs

```
AnharmonicPartitionDerivGeneralH → AnharmonicPartitionHigherDeriv →
AnharmonicGibbsObservableMonomials → AnharmonicGibbsRegularity.
```

Deleting the local theorem from the root of that chain would have orphaned the downstream call site; requalifying it to `Common.AmGm.amgm_t_abs_x` fixes it, and because `Common.AmGm` now rides in transitively along the same chain, no extra `import` line is needed there. The build catches exactly this kind of transitive-consumer breakage that grepping a single file would miss.

4 Roadblocks and resolutions

The manifest contamination, and the false start. The first attempt at this rewire (an earlier, abandoned worktree) tripped over `lean-common`’s `lake-manifest.json`: it carried a spurious `Threepoint` package entry left over from an earlier experiment. I initially read a `laplace` build failure as “the stray entry contaminates the consumer” and abandoned the worktree to fix the manifest first. The re-diagnosis matters: `laplace` *legitimately* depends on `Threepoint` — `AnharmonicGibbsRegularity` imports `Threepoint.CrossSusceptibility` — so the stray entry was a *confounded* test on this particular consumer. The manifest fix (`lean-common` commit `d6ea20e`, regenerated via `lake update` to drop the `Threepoint` entry and preserve the Mathlib pin) is still correct hygiene — it matters for a consumer like `modes-lean` that does *not* independently depend on `Threepoint` — but it was not what was blocking `laplace`. With the fixed manifest on the remote at `511ee60`, the second attempt resolved and built without incident.

The push “rejection” that wasn’t. Pushing `lean-common` reported “cannot lock ref: is at `511ee60` but expected `0c98b51`” — a benign artifact of a concurrent auto-sync having already advanced the remote to the same commit I was pushing. `git ls-remote` confirmed the remote `main` was already at `511ee60` with the fixed manifest and all modules. Nothing to do.

5 What was reuse, what was new

Nothing here is new mathematics. The lemma was written and proven in a prior tide; this tide is pure plumbing. The only “new” artifacts are the `[[require]]` stanza and the two-character-class

edit (delete + qualify). The value is entirely in de-duplication and in proving the rewiring recipe works, so the next two consumers are mechanical.

6 Lessons

- **Additive-then-rewire is the right promotion cadence under concurrency.** Land the shared module as a file nothing imports; defer the dependency-add and local-copy-drop to a separate, deliberate tide. The first half is unbreakable; the second half is where the build does its checking.
- **Diagnose contamination against an *unconfounded* consumer.** A manifest defect that “contaminates consumers” must be tested on a consumer that doesn’t independently have the dep. Laplace’s own Threepoint dependency masked the real signal; the fix was still right, but for the other consumers, not this one.
- **Grep the file, but trust the build for transitive consumers.** Call sites of a to-be-moved lemma can live downstream of the edited file through the import graph. Requalifying them is enough *if* the moved module rides in along the same chain; the full build is what confirms it.
- **Match Mathlib pins before adding a cross-repo git dependency.** The single failure mode that makes `lake update` unresolvable is a Mathlib commit mismatch between consumer and dependency. Check it first; here both sat at `8a178386`.

7 Follow-ups

- `modes-lean` $\rightarrow$ `Common.Majorization (drop local weighted_sum_le)` and `Common.MoorePenrose (drop local, bridge  $T \leftrightarrow H$ )`.
- `threepoint` $\rightarrow$ `Common.MoorePenrose (drop local, H form)`.
- Once all three land, the cross-repo Moore–Penrose / AM–GM / majorization duplication is fully retired into `lean-common`, and the consolidation arc the session opened is closed.